

Lab2

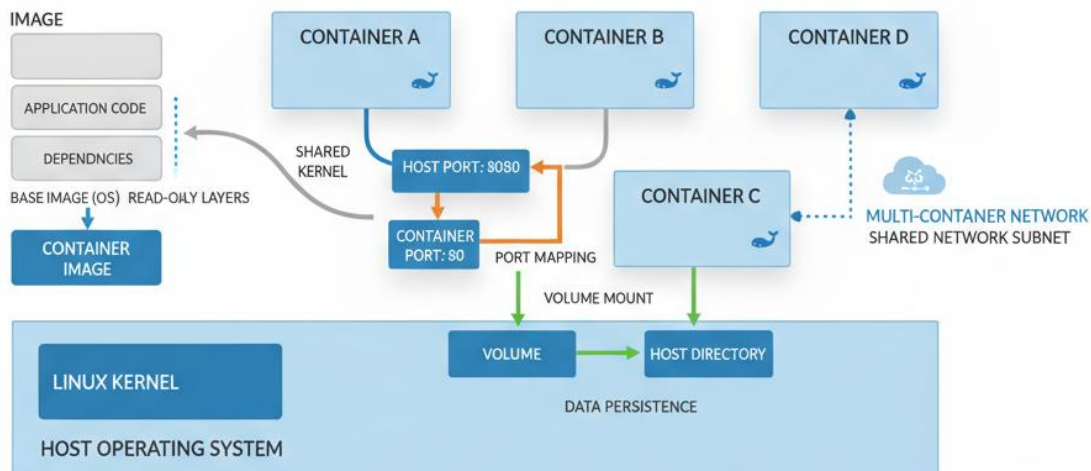
name: WangShiyu nuist_id: 202383910020 setu_id: W20109917

1. Experiment Purpose

- (1) **Understand Docker Core Concepts:** Gain practical knowledge of containers, images, Docker Compose, and their roles in modern application deployment.
- (2) **Master Container Operations:** Learn how to run, manage, and orchestrate containers using both Docker CLI commands and Docker Desktop GUI.
- (3) **Explore Networking and Storage:** Understand how to publish ports, create custom networks, and implement data persistence using volumes and bind mounts.
- (4) **Practice Multi-Container Applications:** Deploy complex applications consisting of multiple interconnected services using Docker Compose.
- (5) **Develop Best Practices:** Learn industry-standard practices for containerization, including creating custom images, managing container lifecycles, and ensuring data persistence.

2. Experiment Process

Overall Process:



Part 0: Preparatory steps

```
PS C:\Users\WangShiyu> docker --version
Docker version 29.3.1, build c2be9cc
PS C:\Users\WangShiyu> docker compose version
Docker Compose version v5.1.1
```

Part 1: Fundamental Concepts

Step 1: What is a Container

1. Concept Understanding

A container is a lightweight, standalone, executable package that includes everything needed to run a piece of software, including the code, runtime, system tools, libraries, and settings. Unlike

virtual machines that require a full operating system, containers share the host system's kernel and isolate the application processes from each other.

Key characteristics of containers:

Characteristic	Description
Self-contained	Includes all dependencies needed to run
Isolated	Processes are separated from host and other containers
Independent	Can be started, stopped, moved independently
Portable	Runs identically across different environments
Lightweight	Shares host kernel, minimal overhead

2. Practise(2 methods)

CLI Command	<pre>PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker run -d -p 8080:80 docker/welcome-to-docker 706ecef41c214291509cd8aedec3565b6a4f79e861c53c1fff13bb344f7b431c PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker ps</pre> <table><thead><tr><th>CONTAINER ID</th><th>IMAGE</th><th>COMMAND</th><th>CREATED</th><th>STATUS</th><th>PORTS</th></tr></thead><tbody><tr><td>706ecef41c21</td><td>docker/welcome-to-docker</td><td>"/docker-entrypoint..."</td><td>7 seconds ago</td><td>Up 6 seconds</td><td>0.0.0.0:8080->80/tcp,</td></tr><tr><td>[::]:8080->80/tcp</td><td>jolly_torvalds</td><td></td><td></td><td></td><td></td></tr></tbody></table> ● docker run: Create and start a new container from an image ● -d: Detached mode - run container in background ● -p 8080:80: Map host port 8080 to container port 80 ● docker/welcome-to-docker: Image name from Docker Hub	CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	706ecef41c21	docker/welcome-to-docker	"/docker-entrypoint..."	7 seconds ago	Up 6 seconds	0.0.0.0:8080->80/tcp,	[::]:8080->80/tcp	jolly_torvalds				
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS														
706ecef41c21	docker/welcome-to-docker	"/docker-entrypoint..."	7 seconds ago	Up 6 seconds	0.0.0.0:8080->80/tcp,														
[::]:8080->80/tcp	jolly_torvalds																		
Docker Desktop GUI	1) Open Docker Desktop and type “welcome-to-docker” in the top search bar. 2) Click the “Pull” button to download the image, after the download completes, click “Run”. welcome-to-docker Gordon Images (51) Containers (0) Volumes (0) Extensions (0) Docs (0) Hub images (50) Hub private repos (0) Local images (1) doc to- docker docker/welcome-to-docker 5M+ · 72 latest Pull Run docker/welcome-to-docker:latest 22.17 MB mickeygor/welcome-to-docker 6.5K · 0 Encountered network issues during the download, adding mirror sites successfully resolved them. Settings Give feedback Search settings General Resources Docker Engine Builders AI Kubernetes Software updates Extensions Beta features Notifications Docker Engine v29.3.1 Configure the Docker daemon by typing a json Docker daemon configuration file. This can prevent Docker from starting. Use at your own risk. <pre>{ "defaultKeepStorage": "2000", "enabled": true }, "experimental": false, "registry-mirrors": ["https://docker.ipanel.live", "https://docker.m.daocloud.io", "https://hub.rat.dev", "https://docker.hiyou.org", "https://docker.rainbond.cc"] }</pre> 3) Expand the optional settings: ● Container name: welcome-to-docker ● Host port: 8080																		



Run a new container
docker/welcome-to-docker:latest

Optional settings

Container name

welcome-to-docker

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port

8080

:80/tcp

Volumes

Host path

...

Container path

+

Environment variables

Variable

Value

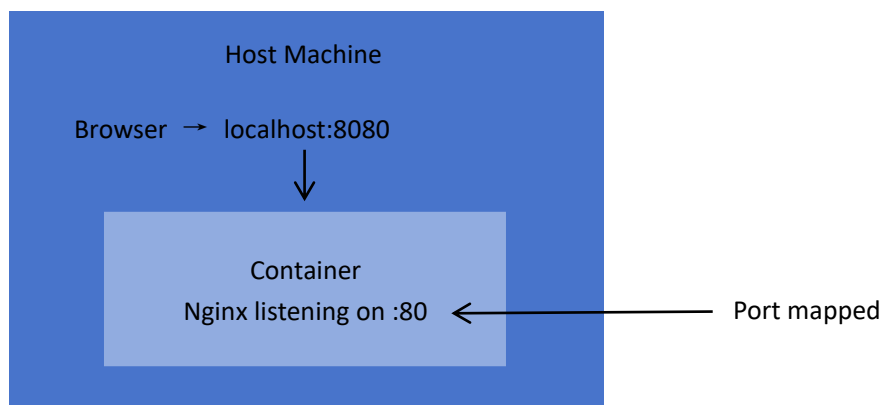
+

Cancel

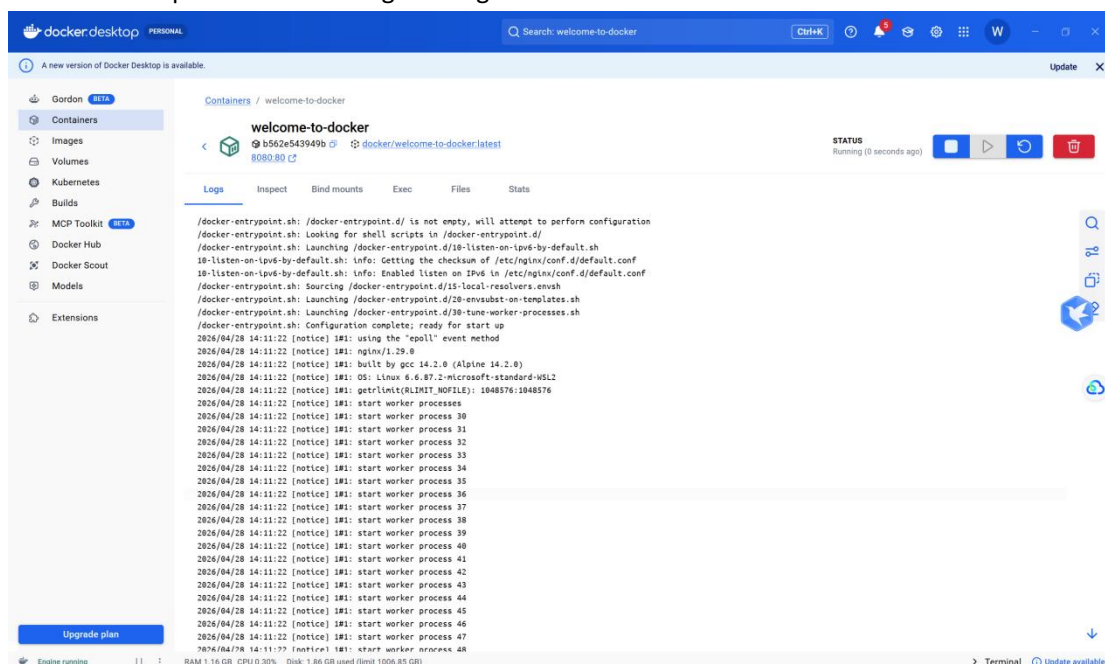
Run

4) Click "Run" to start the container.

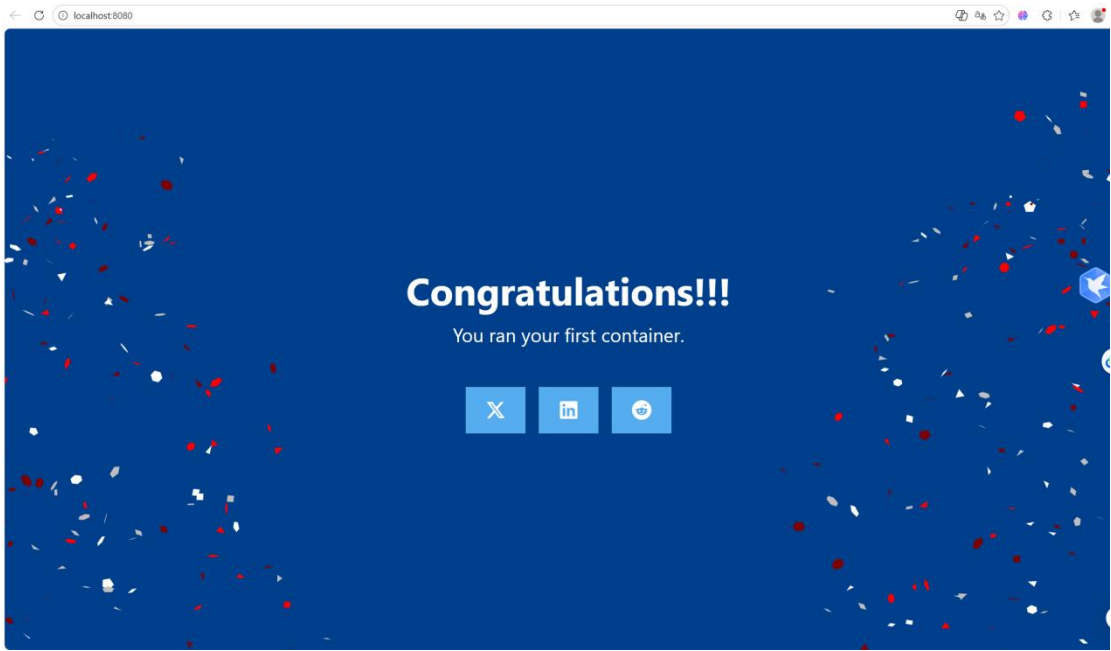
Upon execution, Docker downloaded the welcome-to-docker image and started a container. Accessing <http://localhost:8080> displayed the Docker welcome page, confirming successful container deployment.



Docker Desktop interface showing running containers.



Browser page displaying http://localhost:8080



Step 2: Understanding Images

1. Concept Understanding

A Docker image is a read-only template used to create containers. Images are built in layers, where each layer represents a set of filesystem changes. This layered architecture enables efficient storage, fast builds through caching, and easy distribution.

Two Fundamental Principles:

- Immutability: Once an image is created, it cannot be modified. Any changes result in a new image. This ensures consistency and reproducibility across different deployments.
- Layered Structure: Images are composed of multiple read-only layers stacked on top of each other. Each instruction in a Dockerfile creates a new layer. This design enables:
 - Efficient storage (shared layers between images)
 - Faster builds (layer caching)
 - Smaller image transfers (only new layers are pulled)

2. Practise(2 methods)

Docker images use a layered filesystem (UnionFS), where each layer represents a set of filesystem changes. This design enables efficient storage and faster image builds through layer caching.

Using CLI	Images are read-only templates used to create containers. <pre>P5 C:\Users\MangShiyu\todo-list-app\public_html\nginx-node-redis> docker pull docker/welcome-to-docker Using default tag: latest latest: Pulling from docker/welcome-to-docker Digest: sha256:c4d56c24da4f089ecf8352146b43497fe78953edb4c679b841732beb97e588b0 Status: Image is up to date for docker/welcome-to-docker:latest docker.io/docker/welcome-to-docker:latest</pre> ● docker pull: downloads images from Docker Hub. ● docker image history: reveals the layered structure of images, showing how each instruction in a Dockerfile creates a new layer.
Using Docker Desktop GUI	1) Open Docker Desktop, select “Images” view from the left sidebar, and click “Search images to run”.

2) Search for “welcome-to-docker”, select the “docker/welcome-to-docker” image, and then click “Pull” to download

3) Click on the image to view details, including:

- Image layers
- Installed packages and libraries
- Security vulnerability information

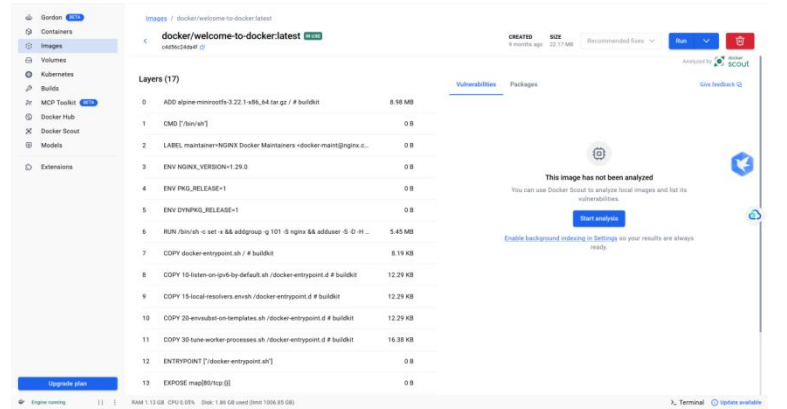
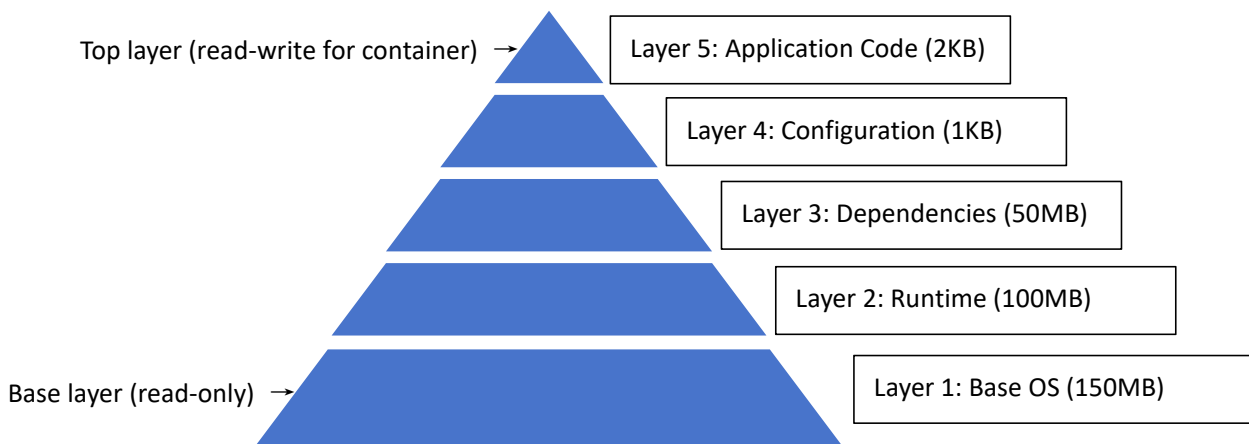


Image Layer Structure:



docker images:

```
PS C:\Users\WangShiyu> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
docker.gitea.com/gitea:1.26.1	d8667667b4cc	245MB	69.3MB	U
docker/welcome-to-docker:latest	c4d56c24da4f	22.2MB	6.03MB	U

docker image history:

```
PS C:\Users\WangShiyu> docker image history docker/welcome-to-docker
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
c4d56c24da4f	9 months ago	COPY /app/build /usr/share/nginx/html # buil...	1.65MB	buildkit.dockerfile.v0
<missing>	10 months ago	CMD ["nginx" "-g" "daemon off;"]	0B	buildkit.dockerfile.v0
<missing>	10 months ago	STOPSIGNAL SIGQUIT	0B	buildkit.dockerfile.v0
<missing>	10 months ago	EXPOSE map[80/tcp:{}]	0B	buildkit.dockerfile.v0
<missing>	10 months ago	ENTRYPOINT ["/docker-entrypoint.sh"]	0B	buildkit.dockerfile.v0
<missing>	10 months ago	COPY 30-tune-worker-processes.sh /docker-ent...	16.4kB	buildkit.dockerfile.v0
<missing>	10 months ago	COPY 20-envsubst-on-templates.sh /docker-ent...	12.3kB	buildkit.dockerfile.v0
<missing>	10 months ago	COPY 15-local-resolvers.envsh /docker-entryp...	12.3kB	buildkit.dockerfile.v0
<missing>	10 months ago	COPY 10-listen-on-ipv6-by-default.sh /docker...	12.3kB	buildkit.dockerfile.v0
<missing>	10 months ago	COPY docker-entrypoint.sh / # buildkit	8.19kB	buildkit.dockerfile.v0
<missing>	10 months ago	RUN /bin/sh -c set -x && addgroup -g 101...	5.45MB	buildkit.dockerfile.v0
<missing>	10 months ago	ENV DYNPKG_RELEASE=1	0B	buildkit.dockerfile.v0
<missing>	10 months ago	ENV PKG_RELEASE=1	0B	buildkit.dockerfile.v0
<missing>	10 months ago	ENV NGINX_VERSION=1.29.0	0B	buildkit.dockerfile.v0
<missing>	10 months ago	LABEL maintainer=NGINX Docker Maintainers <d...	0B	buildkit.dockerfile.v0
<missing>	10 months ago	CMD ["/bin/sh"]	0B	buildkit.dockerfile.v0
<missing>	10 months ago	ADD alpine-minirootfs-3.22.1-x86_64.tar.gz /...	8.98MB	buildkit.dockerfile.v0

Step 3: What is Docker Compose

1. Concept Understanding

Docker Compose allows you to define multiple containers and their configurations in a single YAML file

Best practice: each container should do only one thing

Dockerfile vs Compose file:

- Dockerfile: builds images
- Compose file: defines running containers

2. Practise

Docker Compose is a tool for defining and running multi-container applications.

1) Clone Sample Application

```
PS C:\Users\WangShiyu> git clone https://github.com/docker-samples/todo-list-app
Cloning into 'todo-list-app'...
remote: Enumerating objects: 96, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 96 (delta 5), reused 2 (delta 2), pack-reused 78 (from 1)
Receiving objects: 100% (96/96), 1.68 MiB | 17.00 KiB/s, done.
Resolving deltas: 100% (18/18), done.
PS C:\Users\WangShiyu> cd todo-list-app
```

- git clone: Download the sample application code from GitHub

2) View compose.yaml File

```
PS C:\Users\WangShiyu\todo-list-app> cat compose.yaml
services:
  app:
    image: node:22-alpine
    command: sh -c "yarn install && yarn run dev"
    ports:
      - 127.0.0.1:3000:3000
    working_dir: /app
    volumes:
      - ./:/app
    environment:
      MYSQL_HOST: mysql
      MYSQL_USER: root
      MYSQL_PASSWORD: secret
      MYSQL_DB: todos

  mysql:
    image: mysql:8.0
    volumes:
      - todo-mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: todos

volumes:
  todo-mysql-data:
```

The compose.yaml file defines services, networks, and volumes in a declarative format, which includes:

- Service definitions (app and mysql)
- Network configuration
- Volume configuration

3) Start Application

```
PS C:\Users\WangShiyu\todo-list-app> docker compose up -d --build
[+] up 23/24
✔Image node:22-alpine           Pulled                               1214.0s
✔Image mysql:8.0                Pulled                               1607.5s
✔Network todo-list-app_default   Created                             0.1s
✔Volume todo-list-app_todo-mysql-data Created                             0.0s
✔Container todo-list-app-mysql-1 Started                               1.2s
- Container todo-list-app-app-1 Starting                               1.2s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint todo-list-app-app-1 (e688ad2fb128ec2e994b6e6baaa3b193b4c48fe67efb58efbb20464118411407): Bind for 0.0.0.0:3000 failed: port is already allocated
```

- docker compose up: Create and start containers defined in compose.yaml.
- up -d --build: builds images if needed and starts containers in detached mode.
- -d: Run containers in detached mode (background)
- --build: Build images before starting containers

Encountered network issues: the container cannot connect to the internet to download dependencies; DNS configuration takes effect.

```
"dns": [
  "8.8.8.8",
  "8.8.4.4",
  "114.114.114.114"
],
```

Configuration complete, and the application started successfully.

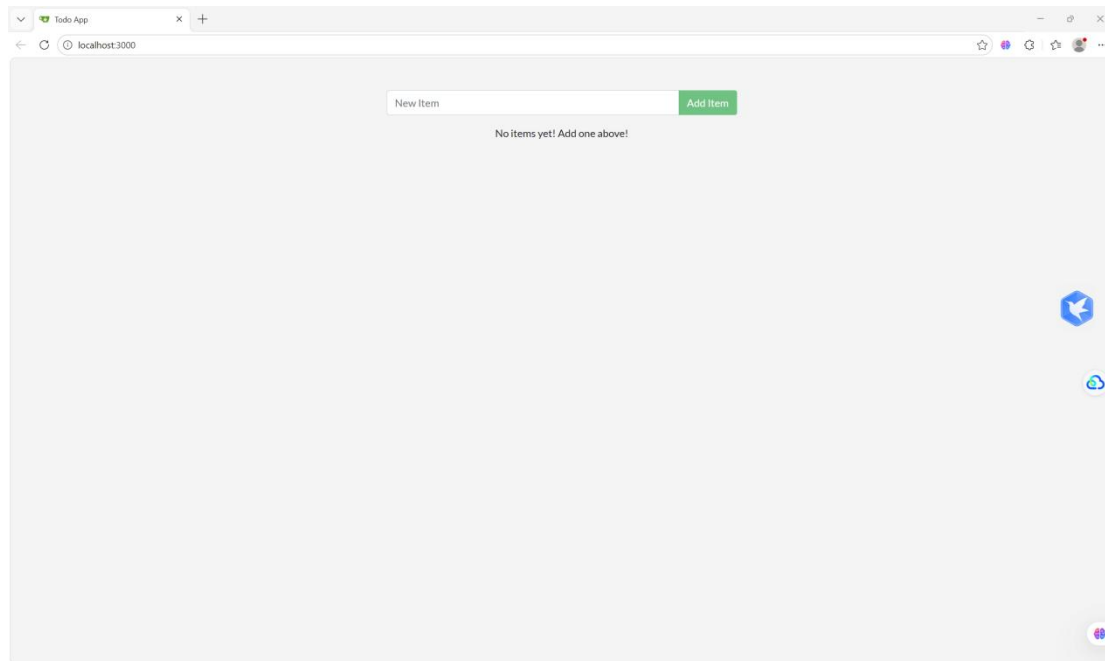
```
PS C:\Users\WangShiyu\todo-list-app> docker compose up
[+] up 3/3
✓Network todo-list-app_default Created 0.0s
✓Container todo-list-app-mysql-1 Created 0.2s
✓Container todo-list-app-app-1 Created 0.2s
Attaching to app-1, mysql-1
mysql-1 | 2026-04-28 15:26:21+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
mysql-1 | 2026-04-28 15:26:21+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
mysql-1 | 2026-04-28 15:26:21+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1.el9 started.
mysql-1 | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
app-1 | yarn install v1.22.22
mysql-1 | 2026-04-28T15:26:22.255192Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and
will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
mysql-1 | 2026-04-28T15:26:22.256752Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.46) starting as proc
ess 1
mysql-1 | 2026-04-28T15:26:22.262549Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
app-1 | success Already up-to-date.
app-1 | Done in 0.41s.
app-1 | yarn run v1.22.22
app-1 | $ nodemon -L src/index.js
mysql-1 | 2026-04-28T15:26:22.788327Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
mysql-1 | 2026-04-28T15:26:23.112967Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
mysql-1 | 2026-04-28T15:26:23.113043Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Enc
rypted connections are now supported for this channel.
mysql-1 | 2026-04-28T15:26:23.118028Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --pid-file: Location
'/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a different directory.
mysql-1 | 2026-04-28T15:26:23.136345Z 0 [System] [MY-011323] [Server] X Plugin ready for connections. Bind-address: '::
' port: 33060, socket: /var/run/mysqld/mysqld.sock
mysql-1 | 2026-04-28T15:26:23.136486Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version:
'8.0.46' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
app-1 | [nodemon] 2.0.20
app-1 | [nodemon] to restart at any time, enter 'rs'
app-1 | [nodemon] watching path(s): *.*
app-1 | [nodemon] watching extensions: js,mjs,json
app-1 | [nodemon] starting 'node src/index.js'
app-1 | Waiting for mysql:3306.
app-1 | Connected!
app-1 | Connected to mysql db at host mysql
app-1 | Listening on port 3000
```

Both services were connected through a custom network, with persistent storage for the database.

- Downloads node and MySQL images
- Creates network
- Creates volumes
- Starts containers

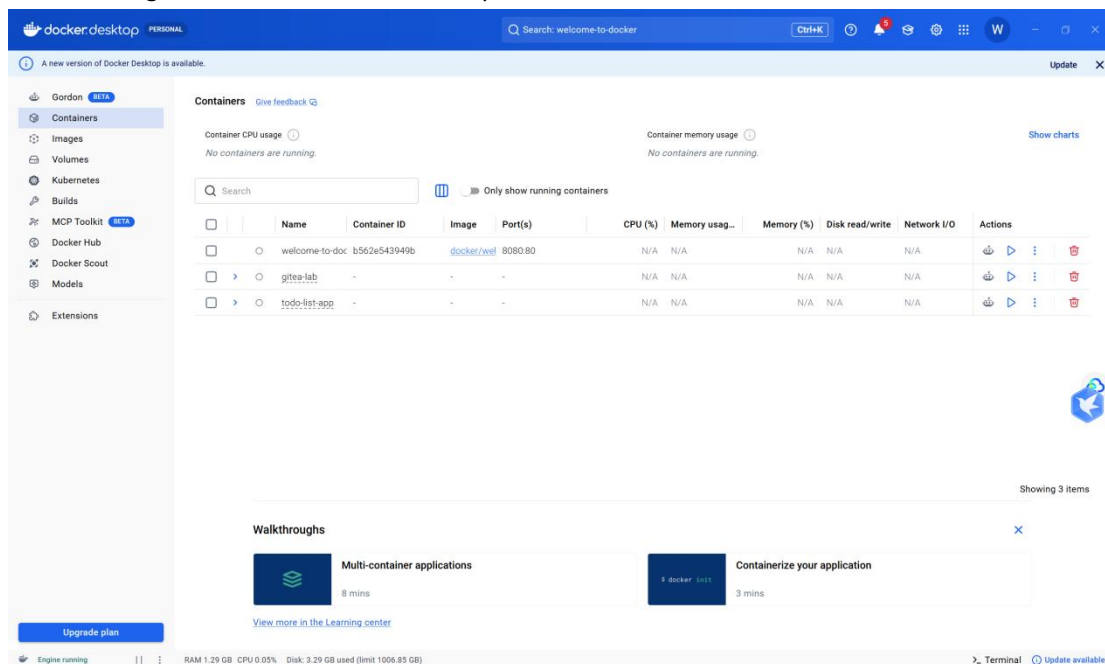
4) Access Application

Open browser and visit: <http://localhost:3000>, and then wait 10-15 seconds for the application to fully start.



5) View Containers

View running containers in Docker Desktop.



Part 2: Running Containers

Step 4: Publishing Ports

1. Concept Understanding

Port publishing is the mechanism that allows external access to services running inside containers. Since containers have their own isolated network namespaces, services inside containers are not directly accessible from the host or external networks without explicit port mapping.

Port Mapping Syntax:

Syntax	Description
-p 8080:80	Map host port 8080 to container port 80
-p 192.168.1.100:8080:80	Bind to specific host IP
-p 80	Let Docker assign a random host port
-P	Publish all exposed ports to random host ports

2. Practise

Port publishing exposes container ports to the host system.

1) Run Container and Publish Port

```
PS C:\Users\WangShiyu\todo-list-app> docker run -d -p 8080:80 --name nginx-server nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
801a1ad15b4e: Pull complete
ce776bbcdad0d: Pull complete
3531af2bc2a9: Pull complete
4677c2a9a3d4: Pull complete
677c63196868: Pull complete
ff048f1f2159: Pull complete
85c66128325a: Pull complete
54c38c75806e: Download complete
69989ccd189b: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
0d1a0be168dcb881e006078586f4bdf7563eadaa4590cd5c562060b656c30bda
```

The syntax -p HOST_PORT:CONTAINER_PORT maps a specific host port to a container port, which allows external access to services running inside containers.

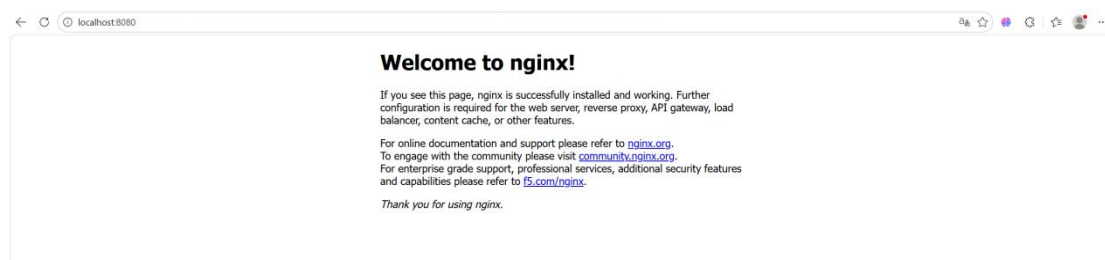
2) Verify Port Publishing

Check the PORTS column showing "0.0.0.0:8080->80/tcp"

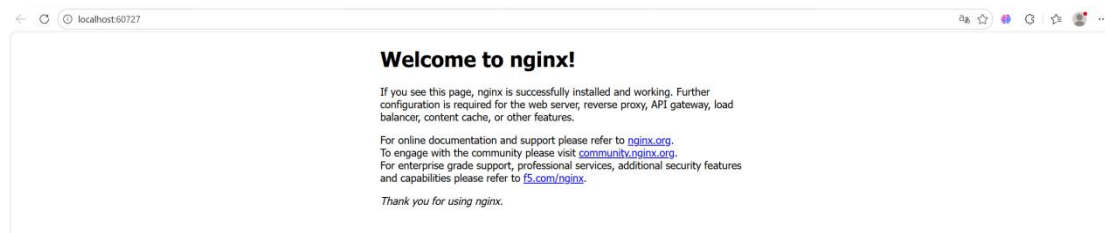
```
PS C:\Users\WangShiyu\todo-list-app> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS
NAME: nginx-server
0d1a0be168dc   nginx    "/docker-entrypoint..."  21 seconds ago  Up 20 seconds  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

3) Access Application

Visit in browser: <http://localhost:8080>



Visit in browser: <http://localhost:60727>

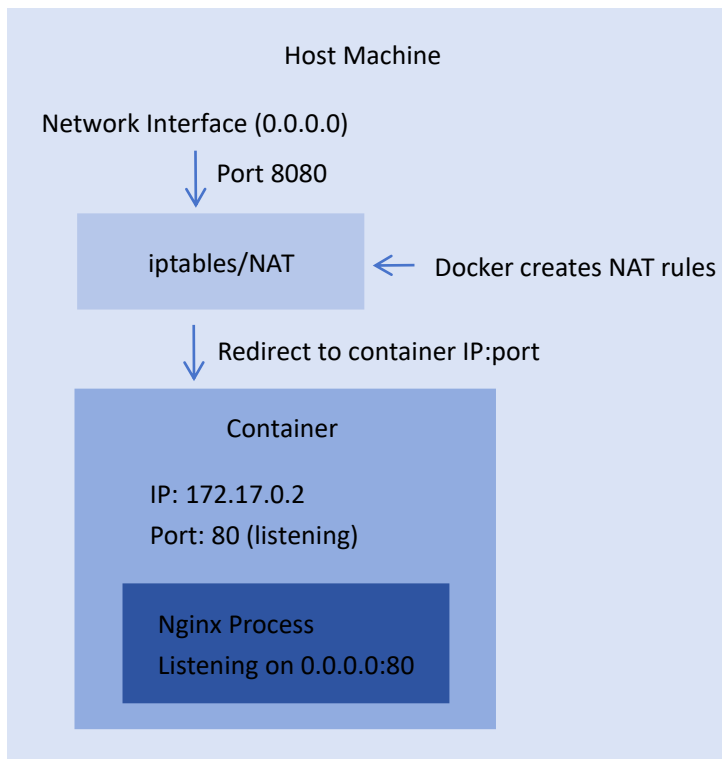


4) Use Random Port

```

PS C:\Users\WangShiyu\todo-list-app> docker run -d -p 80 --name nginx-random nginx
4f555b668a37d039693d6b4f1f3664b674777d448bb09a18e42fe00a9962c3ac
PS C:\Users\WangShiyu\todo-list-app> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
4f555b668a37   nginx    "/docker-entrypoint..." 12 seconds ago Up 11 seconds  0.0.0.0:60727->80/tcp, [::]:6
0727->80/tcp
0d1a0be168dc   nginx    "/docker-entrypoint..."  About a minute ago Up About a minute  0.0.0.0:8080->80/tcp, [::]:80
80->80/tcp
nginx-server

```



The 0.0.0.0:8080->80/tcp indicates:

- 0.0.0.0:8080: Host is listening on all network interfaces on port 8080
- ->80/tcp: Traffic is forwarded to container port 80
- Tcp: Protocol being used

Using -p 80 without specifying a host port lets Docker automatically assign an available high-numbered port.

Step 5: Overriding Container Defaults

1. Concept Understanding

Docker containers come with default configurations defined in their images. However, Docker provides extensive flexibility to override these defaults at runtime, allowing customization without modifying the image.

Common override options:

- Port mapping (“-p”)
- Environment variables (“-e”)
- Resource limits (“--memory”, “--cpus”)
- Network (“--network”)

2. Practise

Multiple containers of the same image can run simultaneously by using different names and port mappings. Custom networks enable isolated communication between containers.

1) Run Multiple PostgreSQL Instances

Running multiple database instances is common in production scenarios.

- Development/Testing: Different databases for different purposes
- Version Migration: Running different PostgreSQL versions side-by-side
- Load Distribution: Distributing read queries across replicas

First instance:

```
PS C:\Users\WangShiyu\todo-list-app> docker run -d -p 5432:5432 --name postgres1 -e POSTGRES_PASSWORD=secret postgres
Unable to find image 'postgres:latest' locally
latest: Pulling from library/postgres
e57497b6cf23: Pull complete
85f6ddcd85b0: Pull complete
c43308a19d8e: Pull complete
b7036600a44b: Pull complete
2428a49310a9: Pull complete
6fadfe1078a3: Pull complete
4e2103318c3c: Pull complete
56b9e522a2ac: Pull complete
13f7f91984c8: Pull complete
c3f159f3b340: Pull complete
ce10992ea794: Pull complete
e9cad8a9e695: Pull complete
6e695e05adc6: Download complete
631e25198ba2: Download complete
Digest: sha256:78481659c47e862334611ccdaf7c369c986b3046da9857112f3b309114a65fb4
Status: Downloaded newer image for postgres:latest
8a1f708071f6cc7aa1bd7c3c658f7d7ec3b10d57b18392ec03a7c0e35085cb71
```

Second instance (different port):

```
PS C:\Users\WangShiyu\todo-list-app> docker run -d -p 5433:5432 --name postgres2 -e POSTGRES_PASSWORD=secret postgres
696f4fc368be33fe0e60f5e15a61a2604ac383072a94a05922c6b44302697bc5
```

- -e, --env: Set environment variables
- -p, --publish: Publish container ports

2) Verify Both Containers Are Running

696f4fc368be	postgres	"docker-entrypoint.s..."	19 minutes ago	Up 19 minutes	0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp	postgres2
8a1f708071f6	postgres	"docker-entrypoint.s..."	19 minutes ago	Up 19 minutes	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	postgres1

☐		postgres1	8a1f708071f6	postgres	5432:5432	0.07%	26.93MB / 7.57G	0.35%	10.8MB / 41.1ME	998B / 126B				
☐		postgres2	696f4fc368be	postgres	5433:5432	0.11%	23.5MB / 7.57GB	0.3%	0B / 40.9MB	872B / 126B				

3) Use Custom Network

Docker creates a default bridge network, but custom networks provide.

- DNS Resolution: Containers can communicate by name instead of IP
- Isolation: Containers on different networks are isolated
- Easier Management: Group related containers together

```
PS C:\Users\WangShiyu\todo-list-app> docker network create mynetwork
052ae0caac14ae29593eeb107a2c486fc52c209ceba2bbf2d4f99ba8a8facde7
PS C:\Users\WangShiyu\todo-list-app> docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
29dda683e316        bridge              bridge              local
a4ee14260ab1        gitea-lab_gitea     bridge              local
5413ed7e8e22        host                host                local
052ae0caac14        mynetwork           bridge              local
bc8ece7bbce7        none                null                local
6037860161d6        todo-list-app_default bridge              local
PS C:\Users\WangShiyu\todo-list-app> docker run -d -p 5434:5432 --name postgres3 --network mynetwork -e POSTGRES_PASSWORD=secret postgres
af9452b177daefa4cac86669475efb743b36c81f44226c8ff548afb706636146
```

- --network: Connect to a specific network
- --memory: Set memory limit

4) Set Resource Limits

Monitor resource usage, screenshot of docker stats output.

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
3ad254bbedfe	limited-app	0.00%	16.65MiB / 512MiB	3.23%	872B / 126B	1.43MB / 12.3kB	21
af9452b177da	postgres3	0.00%	42.41MiB / 7.568GiB	0.55%	1.7kB / 126B	24.7MB / 42.7MB	9
696f4fc368be	postgres2	0.01%	27.79MiB / 7.568GiB	0.36%	998B / 126B	3.78MB / 42.7MB	9
8a1f708071f6	postgres1	0.02%	31.92MiB / 7.568GiB	0.41%	1.12kB / 126B	16.9MB / 42.7MB	9
4f555b668a37	nginx-random	0.00%	16.57MiB / 7.568GiB	0.21%	5.27kB / 4.01kB	4.11MB / 12.3kB	21
0d1a0be168dc	nginx-server	0.00%	16.18MiB / 7.568GiB	0.21%	8.44kB / 6.27kB	0B / 12.3kB	21

- CPU %: Percentage of host CPU being used
- MEM USAGE / LIMIT: Current memory usage vs maximum allowed
- MEM %: Percentage of allowed memory being used
- NET I/O: Network bytes received/sent
- BLOCK I/O: Disk read/write bytes

Part 3: Data Persistence

Step 6: Data Persistence

1. Concept Understanding

By default, all files created inside a container are stored in the container's writable layer. When the container is deleted, these files are also deleted. This presents a problem for applications that need to persist data, such as databases.

Docker Storage Options:

Storage Type	Managed By	Best For	Location
Volumes	Docker	Persistent data, production	/var/lib/docker/volumes/
Bind Mounts	Host OS	Development, config files	Anywhere on host
tmpfs	Docker	Temporary, sensitive data	Memory only

2. Practise

Volumes are Docker-managed storage that persist data independently of container lifecycles.

1) Create PostgreSQL Container with Volume

```
PS C:\Users\WangShiyu\todo-list-app> docker run -d --name postgres-data -e POSTGRES_PASSWORD=secret -v postgres_data:/var/lib/postgresql postgres
89dea3f844ab06798328ca7e929e5fd73365b202c7333c655ef5373a5a1c9725
```

- The `-v postgres_data:/var/lib/postgresql/data` syntax creates a named volume `postgres_data` and mounts it to the PostgreSQL data directory.
- `postgres_data`: Name of the volume (Docker creates it if it doesn't exist)
- `/var/lib/postgresql/data`: Path inside the container where the volume is mounted

2) Connect to Database and Create Data

```
PS C:\Users\WangShiyu\todo-list-app> docker exec -it postgres-data psql -U postgres
psql (18.3 (Debian 18.3-1.pgdg13+1))
Type "help" for help.

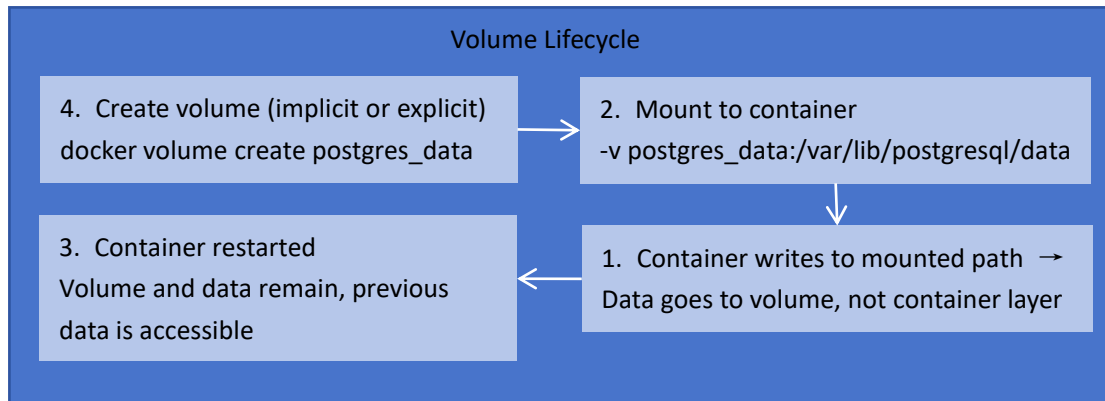
postgres=# \l
postgres=# CREATE DATABASE mydb;
CREATE DATABASE
postgres=# \c mydb
You are now connected to database "mydb" as user "postgres".
mydb=# CREATE TABLE users (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100),
        email VARCHAR(100)
    );
CREATE TABLE
mydb=# INSERT INTO users (name, email) VALUES ('WangShiyu', 'test@example.com');
INSERT 0 1
mydb=# SELECT * FROM users;
 id |  name  | email
----+-----+-----
  1 | WangShiyu | test@example.com
(1 row)

mydb=# \q
```

3) Restart Container and Verify Data Still Exists

```
PS C:\Users\WangShiyu\todo-list-app> docker restart postgres-data
postgres-data
PS C:\Users\WangShiyu\todo-list-app> docker exec -it postgres-data psql -U postgres -d mydb -c "SELECT * FROM users;"
 id | name      | email
-----+-----+-----
  1 | WangShiyu | test@example.com
(1 row)
```

Volume Lifecycle:



4) Managing Volumes

List all volumes and view volume details

```
PS C:\Users\WangShiyu\todo-list-app> docker volume ls
DRIVER    VOLUME NAME
local     7c797b894f7fc62ca99ce971d177cb59c6409c55bc993e60a46094778ae0d88a
local     9cc4a60b2d607a89a850201ccb0a2c13af63c1c9152a32aff3a464426feb2c23
local     a028fe26274343761eddf787a8b92b576676a4dd2bd665030bf37038e003c
local     d6b20002cbb94ecb8c7622658ba7877369d85f6caae74d0b088980b055e94e39
local     postgres_data
local     todo-list-app_todo-mysql-data
PS C:\Users\WangShiyu\todo-list-app> docker volume inspect postgres_data
[
  {
    "CreatedAt": "2026-05-02T03:04:41Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/postgres_data/_data",
    "Name": "postgres_data",
    "Options": null,
    "Scope": "local"
  }
]
```

Delete volume

```
PS C:\Users\WangShiyu\todo-list-app> docker stop postgres-data
postgres-data
PS C:\Users\WangShiyu\todo-list-app> docker rm postgres-data
postgres-data
PS C:\Users\WangShiyu\todo-list-app> docker volume rm postgres_data
postgres_data
PS C:\Users\WangShiyu\todo-list-app> docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container.
Are you sure you want to continue? [y/N] y
Deleted Volumes:
9cc4a60b2d607a89a850201ccb0a2c13af63c1c9152a32aff3a464426feb2c23
a028fe26274343761eddf787a8b92b576676a4dd2bd665030bf37038e003c
d6b20002cbb94ecb8c7622658ba7877369d85f6caae74d0b088980b055e94e39
7c797b894f7fc62ca99ce971d177cb59c6409c55bc993e60a46094778ae0d88a
Total reclaimed space: 123.1MB
```

Step 7: Sharing Local Files

1. Concept Understanding

Bind mounts directly link a host file or directory to a container path. Unlike volumes, bind mounts are not managed by Docker and can be located anywhere on the host filesystem. They are primarily used for development workflows where real-time file synchronization is needed.

Aspect	Volumes	Bind Mounts
Management	Docker managed	User managed
Location	Docker area (/var/lib/docker/volumes/)	Anywhere on host
Portability	High (works anywhere)	Low (host-dependent paths)
Performance	Consistent	May vary (especially on Mac/Windows)
Use Case	Production data	Development, config files

2. Practise

Bind mounts directly link a host directory to a container path, enabling real-time file synchronization. This is particularly useful for development environments where code changes need immediate reflection.

1) Create Local Directory and File

Create directory

```
PS C:\Users\WangShiyu\todo-list-app> mkdir public_html

目录: C:\Users\WangShiyu\todo-list-app

Mode                LastWriteTime         Length Name
----                -
d-----          2026/5/2   12:50             public_html

PS C:\Users\WangShiyu\todo-list-app> cd public_html
```

Create index.html

```
PS C:\Users\WangShiyu\todo-list-app\public_html> Set-Content -Path index.html -Value @'
>> <!DOCTYPE html>
>> <html lang="en">
>> <head>
>>   <meta charset="UTF-8">
>>   <title>My Website with Docker!</title>
>> </head>
>>   <meta charset="UTF-8">
>>   <title>My Website with Docker!</title>
>> </head>
>> <body>
>>   <h1>Welcome!</h1>
>>   <p>This page is served from a bind mount!</p>
>> </body>
>> </html>
>> '@
```

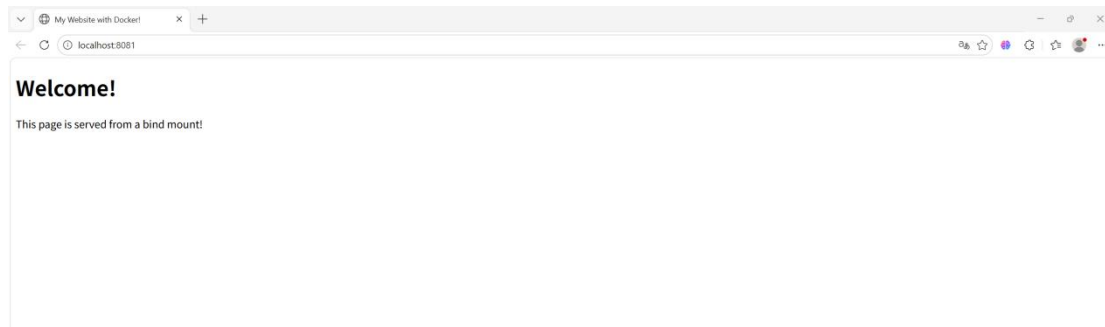
2) Run Container with Bind Mount

```
PS C:\Users\WangShiyu\todo-list-app\public_html> docker run -d --name my_site -p 8081:80 --mount type=bind,source=${PWD},target=/usr/local/apache2/htdocs/ httpd:2.4
6c8f32c87463fa8bdc9751824f35001645015407b18eeddd2823b043d0b5ef43
```

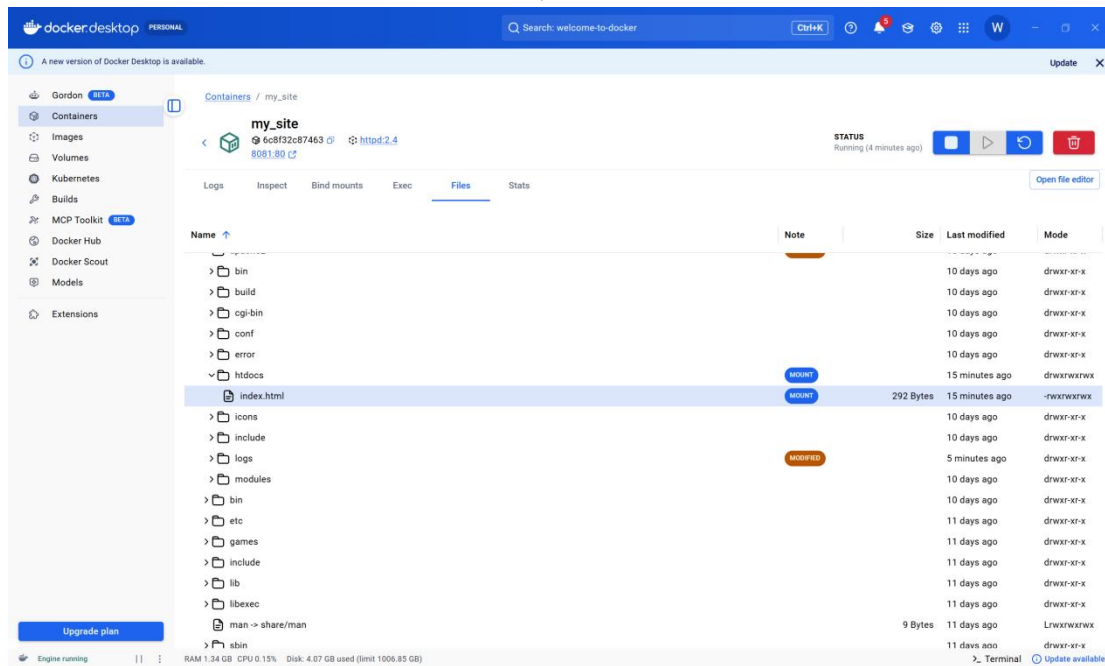
- \${PWD}: Current working directory (PowerShell syntax for present working directory)
- /usr/local/apache2/htdocs/: Default document root for Apache HTTP server

3) Verify File Sharing

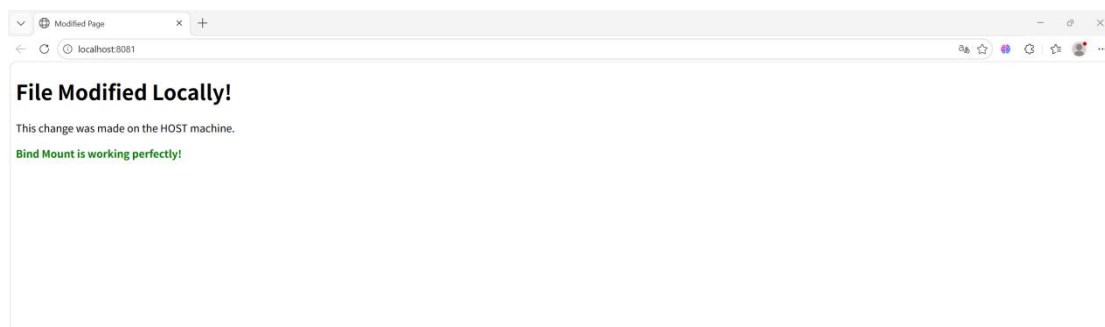
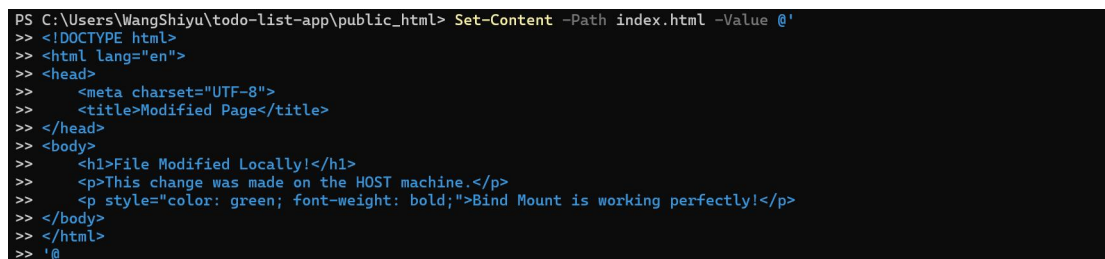
Visit <http://localhost:8081>



View the container's Files tab in Docker Desktop



Modify local "index.html" file, refresh browser to see changes

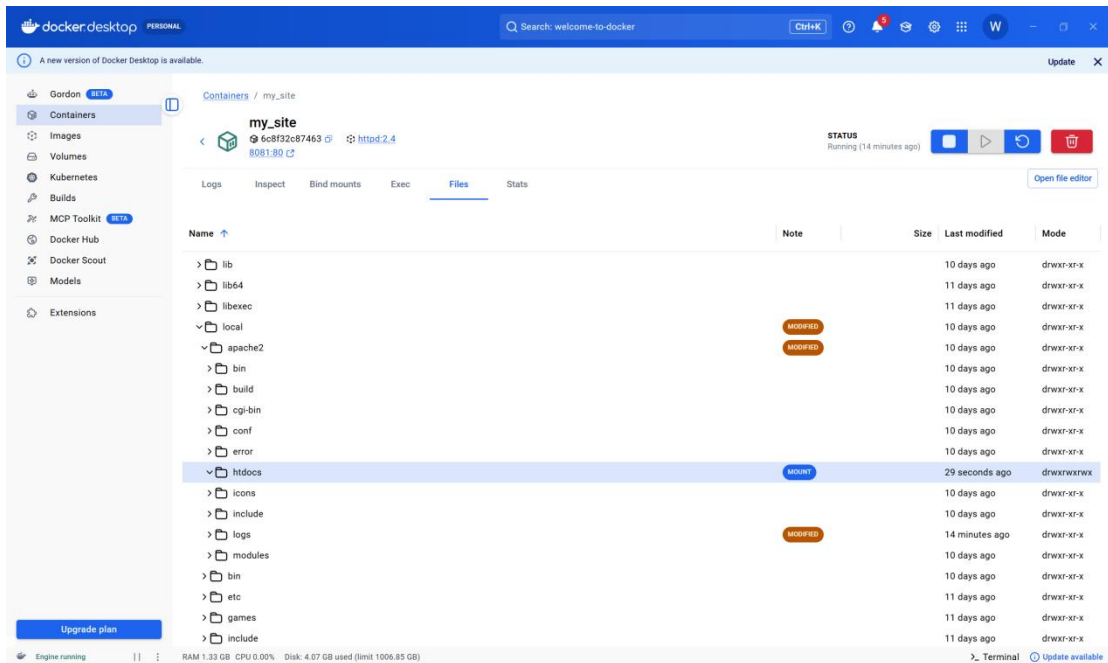


4) Test Bidirectional Synchronization

Delete local file

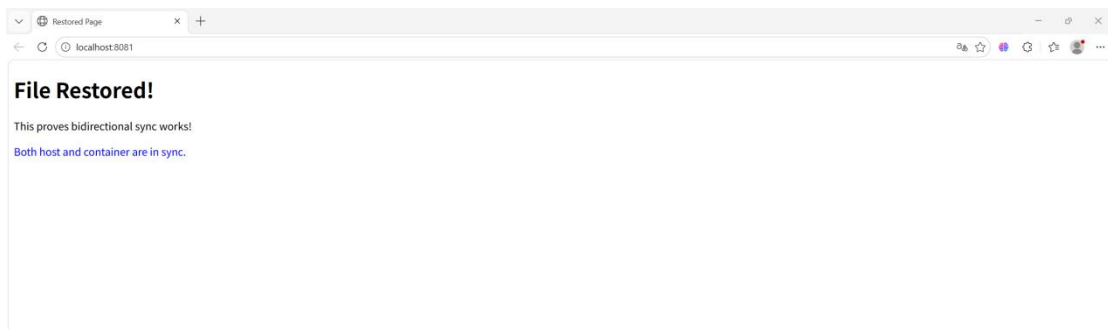
```
PS C:\Users\WangShiyu\todo-list-app\public_html> Remove-Item index.html
PS C:\Users\WangShiyu\todo-list-app\public_html> ls
```

View Files tab in Docker Desktop, find that file is also deleted in container



Create new index.html, observe file restoration in container

```
PS C:\Users\WangShiyu\todo-list-app\public_html> Set-Content -Path index.html -Value @'
>> <!DOCTYPE html>
>> <html lang="en">
>> <head>
>>   <meta charset="UTF-8">
>>   <title>Restored Page</title>
>> </head>
>> <body>
>>   <h1>File Restored!</h1>
>>   <p>This proves bidirectional sync works!</p>
>>   <p style="color: blue;">Both host and container are in sync.</p>
>> </body>
>> </html>
>> '@
```



Bidirectional Synchronization Test:

- (1) **Modified** index.html locally → Changes appeared immediately in container
- (2) **Deleted** index.html locally → File disappeared from container
- (3) **Recreated** index.html locally → File reappeared in container

5) Cleanup

```
PS C:\Users\WangShiyu\todo-list-app\public_html> docker stop my_site
my_site
PS C:\Users\WangShiyu\todo-list-app\public_html> docker rm my_site
my_site
```

Part 4: Multi-Container Applications

Step 8: Multi-Container Application

1. Concept Understanding

Modern applications often consist of multiple services working together. The microservices architecture decomposes applications into small.

- Can be developed and deployed independently
- Scale independently based on demand
- Have clear boundaries and responsibilities
- Communicate through well-defined APIs

2. Practise

1) Clone Sample Application

This command downloads a pre-configured multi-container application from GitHub.

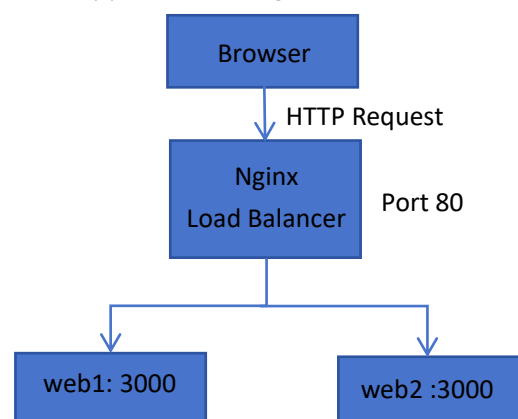
```
PS C:\Users\WangShiyu\todo-list-app\public_html> git clone https://github.com/docker-samples/nginx-node-redis
Cloning into 'nginx-node-redis'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 82 (delta 22), reused 18 (delta 18), pack-reused 54 (from 1)
Receiving objects: 100% (82/82), 76.62 KiB | 160.00 KiB/s, done.
Resolving deltas: 100% (26/26), done.
PS C:\Users\WangShiyu\todo-list-app\public_html> cd nginx-node-redis
```

- nginx/: Nginx configuration and Dockerfile
- web/: Node.js application code and Dockerfile
- docker-compose.yml: Alternative Compose configuration

2) Build Nginx Image

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> cd nginx
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis\nginx> docker build -t nginx .
[+] Building 3.2s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile             0.1s
=> => transferring dockerfile: 141B                             0.0s
=> [internal] load metadata for docker.io/library/nginx:1.29   1.9s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199 0.2s
=> => resolve docker.io/library/nginx:1.29@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199 0.0s
=> [internal] load build context                                0.1s
=> => transferring context: 222B                                  0.0s
=> [2/3] RUN rm /etc/nginx/conf.d/default.conf                 0.3s
=> [3/3] COPY nginx.conf /etc/nginx/conf.d/default.conf        0.1s
=> => exporting to image                                         0.4s
=> => exporting layers                                           0.2s
=> => exporting manifest sha256:3d2e3acc0916ea53150119f767662554545339f77ab4b5df5db472ef220cd787 0.0s
=> => exporting config sha256:585c2b01950f9c77a832b8f8cf1595735482b47a7114797bc4022660a4e61929 0.0s
=> => exporting attestation manifest sha256:76ae8e87022e01c3bbcd065bcf82d5da0e3ad7b31fa31f0cf4d58ca25dec3826 0.0s
=> => exporting manifest list sha256:97cfd6bc00380423263684c92fef09a3fddcdcaa38703398a302e7643eb3c872 0.0s
=> => naming to docker.io/library/nginx:latest                 0.0s
=> => unpacking to docker.io/library/nginx:latest              0.1s
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis\nginx> cd ../web
```

- FROM nginx:1.29: Use official Nginx image as base
- RUN rm ...: Remove default Nginx configuration
- COPY nginx.conf ...: Copy custom configuration for load balancing



3) Build Node.js Image

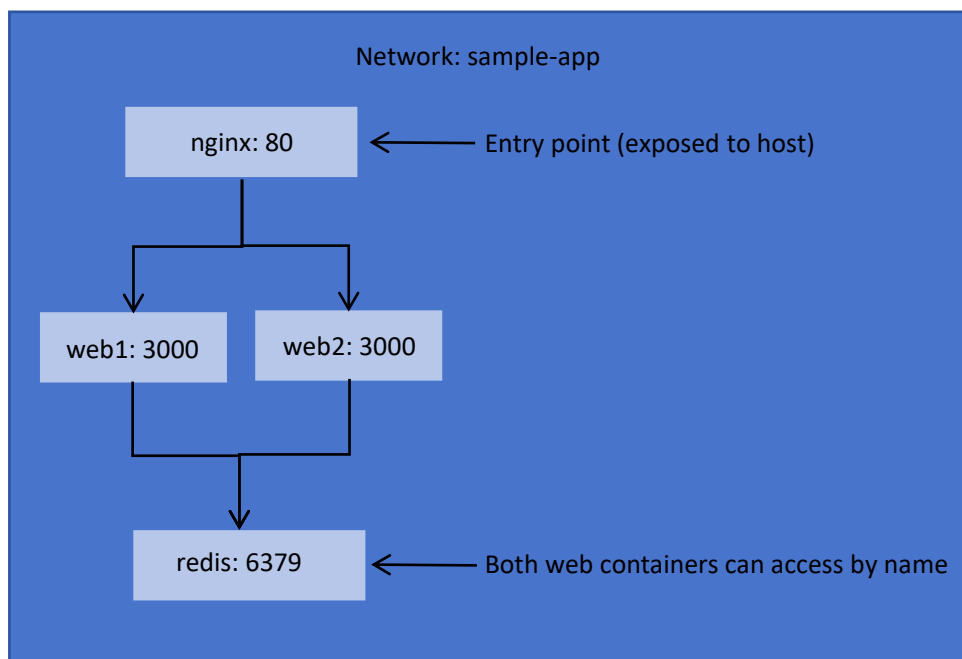
```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis\web> docker build -t web .
[+] Building 166.5s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 175B
=> [internal] load metadata for docker.io/library/node:21
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/5] FROM docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d
=> resolve docker.io/library/node:21@sha256:4b232062fa976e3a966c49e9b6279efa56c8d207a67270868f51b3d155c4e33d
=> sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88 449B / 449B
=> sha256:4684276225df21ffea91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb 49.72MB / 49.72MB
=> sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150 1.25MB / 1.25MB
=> sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2 211.21MB / 211.21MB
=> sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd 3.37kB / 3.37kB
=> sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3 64.14MB / 64.14MB
=> sha256:891494355808bddd3db5552f0d3723fd0fa826675f774853796fafa221d850d42 24.05MB / 24.05MB
=> sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7 49.58MB / 49.58MB
=> extracting sha256:c6cf28de8a067787ee0d08f8b01d7f1566a508b56f6e549687b41dfd375f12c7
=> extracting sha256:891494355808bddd3db5552f0d3723fd0fa826675f774853796fafa221d850d42
=> extracting sha256:6582c62583ef22717db8d306b1d6a0c288089ff607d9c0d2d81c4f8973cbfee3
=> extracting sha256:bf2c3e352f3d2eed4eda4feeed44a1022a881058df20ac0584db70c138b041e2
=> extracting sha256:2b542796ccab17c22fccf6f4a00414f43ecabd5a5d68fe53f8c8c5aee498b8dd
=> extracting sha256:4684276225df21ffea91884cd316f7bc0d99baeef8ff811f98c2ab46188c3cb
=> extracting sha256:08750d53428eebb9ceb8fd6d9516a08420d7747d6a740951c2b043d1ef88c150
=> extracting sha256:5d3106ce01c367e8274a6e83a2d6f3d10ecf9619db69a0b3eacebef36f214f88
=> [internal] load build context
=> transferring context: 17.30kB
=> [2/5] WORKDIR /usr/src/app
=> [3/5] COPY package.json package-lock.json ./
=> [4/5] RUN npm ci
=> [5/5] COPY ./server.js ./
=> exporting to image
=> exporting layers
=> exporting manifest sha256:da2bb186cea414e85dad61153cd11a632f0a32fc5972314a2bd599349b26f12e
=> exporting config sha256:79a961a83478a3b7f05d845fecf5ed15a1e2b753c582dcc183fa0715b872dc48
=> exporting attestation manifest sha256:1c933b392608a8d4fbbf5454cafea07e01c003ee9e2195bf34184002a85c6b6b
=> exporting manifest list sha256:9de5aff8ed33c31cf04a48702c74498b6224566eb8ffc5c24b68b3a2162b3691
=> naming to docker.io/library/web:latest
=> unpacking to docker.io/library/web:latest
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis\web> cd ..
```

- FROM node:21: Use Node.js 21 as base image
- WORKDIR /usr/src/app: Set working directory inside container
- COPY package*.json ./: Copy dependency files first (for caching)
- RUN npm ci: Install dependencies (faster, stricter than npm install)
- COPY server.js ./: Copy application code last

4) Create Docker Network

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker network create sample-app
5c5c496af6f25fa3a26b7825105c7482cdbc5ceb5dfff42d09bde6eda435c13909
```

Network Architecture:



Container Communication:

Container	Can Access	Address
nginx	web1	http://web1:3000
nginx	web2	http://web2:3000
web1	redis	redis://redis:6379
web2	redis	redis://redis:6379

The DNS resolution is automatic because all containers are on the same custom network.

5) Start Redis Container

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker run -d --name redis --network sample-app --network-alias redis redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
2b8951cd65b5: Pull complete
9e0f42ff49e1: Pull complete
4f4fb700ef54: Pull complete
d55df8e0f53d: Pull complete
14d3b17b0010: Pull complete
97afe6025aa9: Pull complete
f426aaaf8949: Download complete
ff0fc1b2ba98: Download complete
Digest: sha256:832d7785830f3f4b559300e6191fc914b15642c1935252338825cf4332200148
Status: Downloaded newer image for redis:latest
8e3a436a8adac4048a128977e09e4206d8ac0c4bc1cc7e2475795a10e21f493b
```

- -d: Run in detached mode (background)
- --name redis: Container name (used for management)
- --network sample-app: Connect to the custom network
- --network-alias redis: DNS alias for the container on this network
- redis: Image name (uses latest tag by default)

6) Start Web Application Containers

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker run -d --name web1 -h web1 --network sample-app --network-alias web1 web
b704fbf64e726a341802d62320623042ebe42ad84abccc23228a4e7bbe613dc7
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker run -d --name web2 -h web2 --network sample-app --network-alias web2 web
d1df19460ac1b32872dba03c5c08f66f76ec44ffbe2e9477d3ca5e4c9639721
```

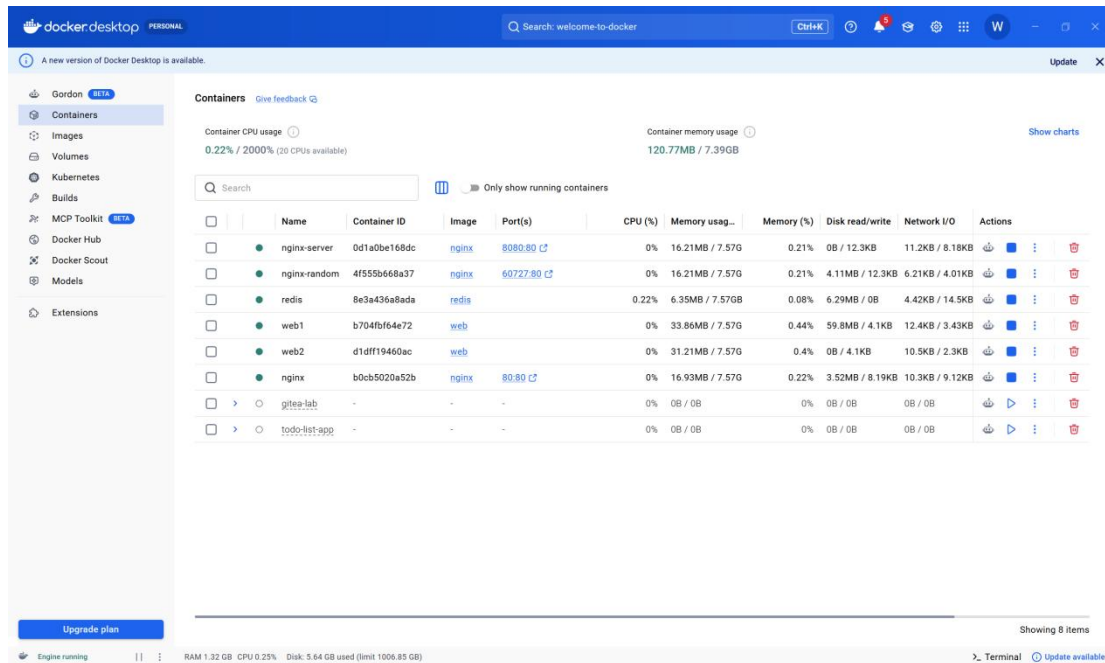
- --name web1: Container name for Docker management
- -h web1: Hostname inside the container
- --network-alias web1: DNS name for network resolution
- web: Custom image we built earlier

7) Start Nginx Container

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker run -d --name nginx --network sample-app -p 80:80 nginx
b0cb5020a52b3490b753371f5a604231ba2f1f48a66d16c352864dc01a64e93b
```

- --network sample-app: Same network as web containers
- -p 80:80: Expose port 80 to host (only entry point)

8) Verify Deployment



9) Test Load Balancing

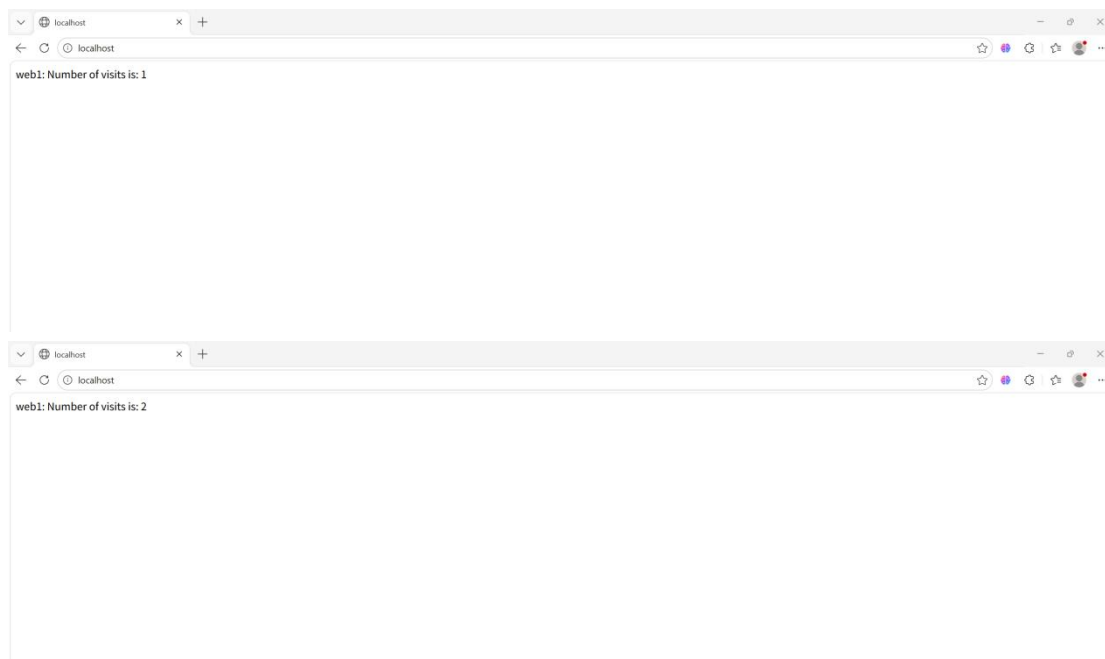
Accessed <http://localhost> in browser and refreshed multiple times.

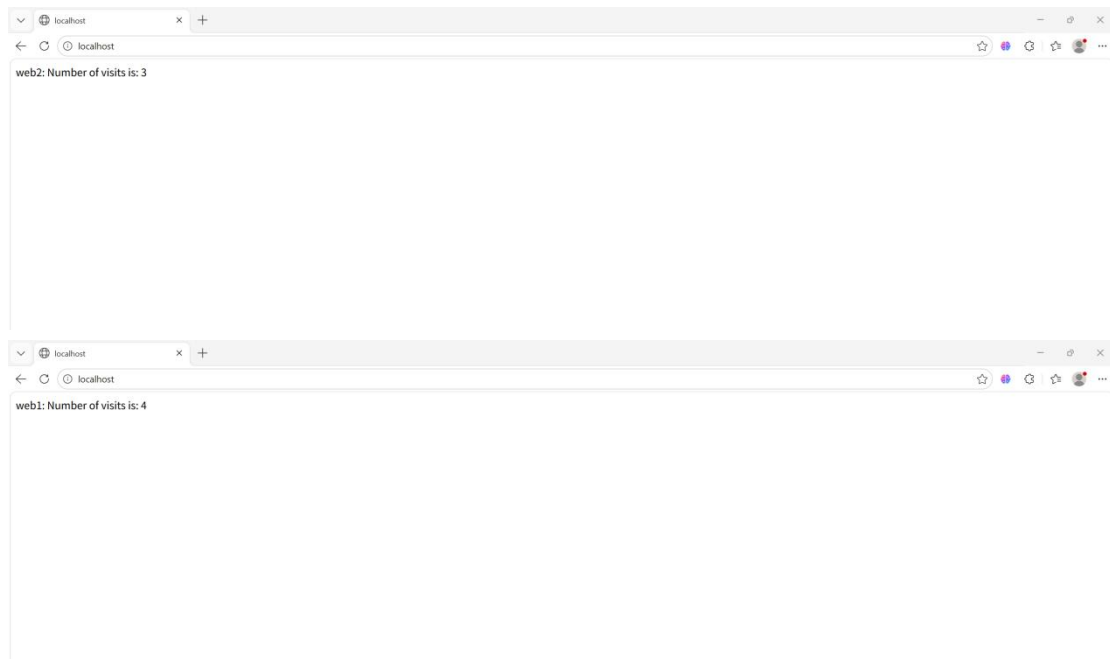
Observed Results:

Count	Content	Container	No Sharing	Sharing
1	web1: Number of visits is: 1	web1	1	1
2	web1: Number of visits is: 2	web1	2	2
3	web2: Number of visits is: 3	web2	1	3
4	web1: Number of visits is: 4	web1	3	4

Load balancing is working properly: requests are distributed to different containers

Shared storage is working properly: access counter keeps incrementing (1→2→3→4)





10) Cleanup Containers

```
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker stop redis web1 web2 nginx
redis
web1
web2
nginx
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker rm redis web1 web2 nginx
redis
web1
web2
nginx
PS C:\Users\WangShiyu\todo-list-app\public_html\nginx-node-redis> docker network rm sample-app
sample-app
```

3. Experiment Result

1) Summary of Deployed Containers

Container Name	Image	Role	Port Mapping	Network
nginx	nginx (custom)	Reverse Proxy & Load Balancer	0.0.0.0:80->80/tcp	sample-app
web1	web (custom)	Application Server	Internal only	sample-app
web2	web (custom)	Application Server	Internal only	sample-app
redis	redis:latest	Data Store	Internal only	sample-app
nginx-server	nginx:latest	Test Server	0.0.0.0:8080->80/tcp	bridge
nginx-random	nginx:latest	Test Server	Random port	bridge

2) Key Achievements

(1) Container Deployment: Successfully deployed 6 containers using both CLI commands and Docker Compose

- (2) Port Publishing: Configured port mappings for external access
- (3) Data Persistence: Verified data persistence using named volumes with PostgreSQL
- (4) File Sharing: Implemented real-time file synchronization using bind mounts
- (5) Custom Networks: Created isolated networks for inter-container communication
- (6) Load Balancing: Deployed a multi-container application with Nginx load balancer

4. Conclusion

1) Container Fundamentals:

- Containers provide isolated, portable runtime environments
- They share the host OS kernel, making them lightweight compared to virtual machines
- Container lifecycles are managed through simple CLI commands

2) Image Management:

- Images are immutable, layered filesystem templates
- Understanding image layers is crucial for optimizing Dockerfile instructions
- Layer caching significantly speeds up image builds

3) Networking:

- Port publishing enables external access to containerized services
- Custom networks provide isolated communication channels
- Network aliases simplify service discovery

4) Storage Options:

- Volumes: Best for persistent data managed by Docker
- Bind Mounts: Ideal for development with real-time file synchronization
- Choosing the right storage strategy depends on the use case

5) Multi-Container Orchestration:

- Docker Compose simplifies multi-container application deployment
- Manual container deployment provides fine-grained control
- Understanding both approaches is valuable for different scenarios

Challenges Encountered and Solutions:

Challenge	Cause	Solution
PowerShell syntax differences	Windows uses different shell syntax	Used Set-Content instead of cat << EOF, \${PWD} instead of \$(pwd)
Docker Hub connection issues	Network restrictions in some regions	Configured Docker mirror registries
Slow first image build	Need to download base images	Subsequent builds use cached images
Port conflicts	Port already in use by another service	Used different host ports or random assignment